ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.XX _____

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №30»

Выполнил:

Иванов Иван Иванович

студент:	XX-XX группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководитель:	*Фамилия Имя Отчество*

Оценка

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	XX-XX
Ф.И.О. студента	**Иванов Иван Иванович**

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №30»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта:

Сценарий	Автомойка
Датчик 1 (аналоговый)	POT (давление воды, 0–10 бар)
Датчик 2	US (расстояние до автомобиля)
Датчик 3	PIR (движение)
Исполнительное устройство	реле (насос)
Светодиод-индикатор (цвет, ток)	белый, 20 мА
Период опроса T	2 с
Порог события	давление < 3 бар
Дата выдачи задания для выполнения курсового проекта	«____» _____ 2026 г.
Срок защиты курсового проекта	«____» _____ 2026 г.

Содержание курсового проекта

Введение

1. Постановка задачи (задание в соответствии с вариантом)
2. Структура и алгоритм работы системы
3. Разработка аппаратной части
4. Разработка прошивки
5. Проектирование базы данных
6. Загрузка и контроль данных в Node-RED
7. Тестирование и результаты
8. Расчёт объёма данных
9. Управление версиями Git

Выводы

Список используемых сокращений

Список используемой литературы

ПРИЛОЖЕНИЯ

А. Структурная схема

Б. Схема электрическая принципиальная

В. Перечень элементов

Г. Модель в Wokwi

Д. Листинг программы Arduino

Е. Листинг функции разбора журнала Node-RED

Ж. Скрипты базы данных

И. Журнал Serial и загрузочный скрипт

К. История Git

Список рекомендованной литературы

Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. – СПб.: БХВ-Петербург, 2016. – 320 с.
Монк С. Программируем Arduino. Профессиональная работа со скетчами. – СПб.: Питер, 2017. – 288 с.
Чакон С., Штрауб Б. Pro Git. – 2-е изд. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 01.10.2026).
ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению. – М.: Стандартинформ, 2009.
Документация SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 01.10.2026).

Руководитель курсового проекта _____ (Фамилия И.О.)
подпись *расшифровка подписи*

Студент _____ (Иванов И.И.)
подпись *расшифровка подписи*

Содержание

Введение 6
1. Постановка задачи 8
2. Структура и алгоритм работы системы 10
2.1. Структурная схема 10
2.2. Алгоритм функционирования 10
2.3. Протокол обмена данными 12
3. Разработка аппаратной части 13
3.1. Выбор компонентов и подключение 13
3.2. Расчёт резисторов и оценка потребления 14
3.3. Принципиальная схема в EasyEDA 15
3.4. Модель в Wokwi 15
4. Разработка прошивки 16
5. Проектирование базы данных 17
5.1. Модель данных 17
5.2. Описание таблиц 17
5.3. Контрольные запросы 18
6. Загрузка и контроль данных в Node-RED 19
6.1. Состав потока 19
6.2. Функция разбора журнала 20
6.3. Запуск 20
7. Тестирование и результаты 21
7.1. Штатный сценарий 21
7.2. Сценарий неисправности датчика 22
7.3. Повторная загрузка 23
7.4. Сводка результатов 23
8. Расчёт объёма данных 24
9. Управление версиями Git 25
Выводы 26
Список используемых сокращений 27
Список используемой литературы 28
Приложение А Структурная схема 29

Приложение Б Схема электрическая принципиальная 30
Приложение В Перечень элементов 31
Приложение Г Модель в Wokwi 32
Приложение Д Листинг программы Arduino (variant_30.ino) 33
Приложение Е Листинг функции разбора журнала Node-RED 35
Приложение Ж Скрипты базы данных 36
Приложение И Журнал Serial и загрузочный скрипт 37
Приложение К История Git 38

Введение

Интеллектуальные интегрированные системы (ИИС) строятся на основе датчиков, исполнительных устройств и программируемых контроллеров, объединённых каналами передачи данных и средствами хранения информации. Типовая задача сопровождения таких систем состоит в том, чтобы получать показания датчиков, сохранять их, находить отклонения от нормы и выдавать результат в виде, пригодном для анализа.

Прежде чем собирать устройство из реальных компонентов, его проектируют и проверяют на модели. Принципиальную схему разрабатывают в системе автоматизированного проектирования EasyEDA, работу программы микроконтроллера проверяют в симуляторе Wokwi, результаты измерений сохраняют в базе данных SQLite, а за загрузку и контроль данных отвечает среда потоковой разработки Node-RED. Все файлы проекта ведутся в системе контроля версий Git. Такой набор программ не требует установки оборудования и позволяет воспроизвести проект на любом компьютере.

Объектом курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Автомойка». Предмет проекта – схемотехника, программа микроконтроллера, структура базы данных и средства загрузки данных, обеспечивающие контроль давления воды, положения автомобиля и движения на посту мойки.

Цель проекта – разработать и документировать работающую виртуальную систему, в которой показания трёх датчиков через последовательный порт попадают в журнал, из журнала автоматически загружаются в базу данных и проверяются на выход за допустимые пределы.

Для достижения цели необходимо решить следующие задачи:

- разработать структурную схему и алгоритм работы системы;
- выбрать компоненты, рассчитать токоограничивающие резисторы и выполнить принципиальную схему в EasyEDA по требованиям ЕСКД;
- собрать модель в Wokwi и написать программу для Arduino Uno;

– спроектировать базу данных SQLite и набор контрольных запросов;
– настроить поток Node-RED, который разбирает журнал, загружает данные в базу и фиксирует неисправности;
– провести тестирование на штатном и аварийном сценариях;
– оформить проект в репозитории Git и подготовить пояснительную записку.

1. Постановка задачи

В соответствии с заданием (вариант №30) разрабатывается система контроля поста автомобильной мойки. Первый датчик измеряет давление воды в магистрали, второй определяет расстояние до автомобиля, третий фиксирует движение в зоне поста. Исполнительным устройством служит реле, включающее насос. Входные данные варианта приведены в таблице 1.

Таблица 1 – Входные данные варианта №30

Параметр	Значение
Сценарий	Автомойка
Датчик 1 (аналоговый), s1	POT – давление воды, диапазон 0–10 бар
Датчик 2, s2	US (HC-SR04) – расстояние до автомобиля, диапазон 2–400 см
Датчик 3, s3	PIR – движение, значения 0 / 1
Исполнительное устройство	Реле K1, управляющее насосом
Светодиод-индикатор	Белый, рабочий ток 20 мА
Период опроса T	2 с
Порог события	Давление s1 < 3 бар

Система должна выполнять следующие функции:

1) каждые T = 2 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA»;
2) при снижении давления ниже 3 бар включать реле насоса и белый светодиод и передавать строку «EVENT» с типом THRESHOLD; при возврате давления в норму выключать их и передавать EVENT с типом NORMAL;
3) сохранять строки последовательного порта в текстовый журнал data/log.txt;
4) автоматически загружать журнал в базу SQLite (таблицы readings и events);
5) при значении датчика вне допустимого диапазона записывать в таблицу событий неисправность FAULT;

6) выдавать три контрольных запроса: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий.

Результатами работы являются: принципиальная схема в EasyEDA, модель в Wokwi, программа Arduino, база данных, поток Node-RED, репозиторий Git и настоящая пояснительная записка с приложениями.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных (рисунок 1). Датчики подключены к входам платы; плата управляет реле насоса и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.

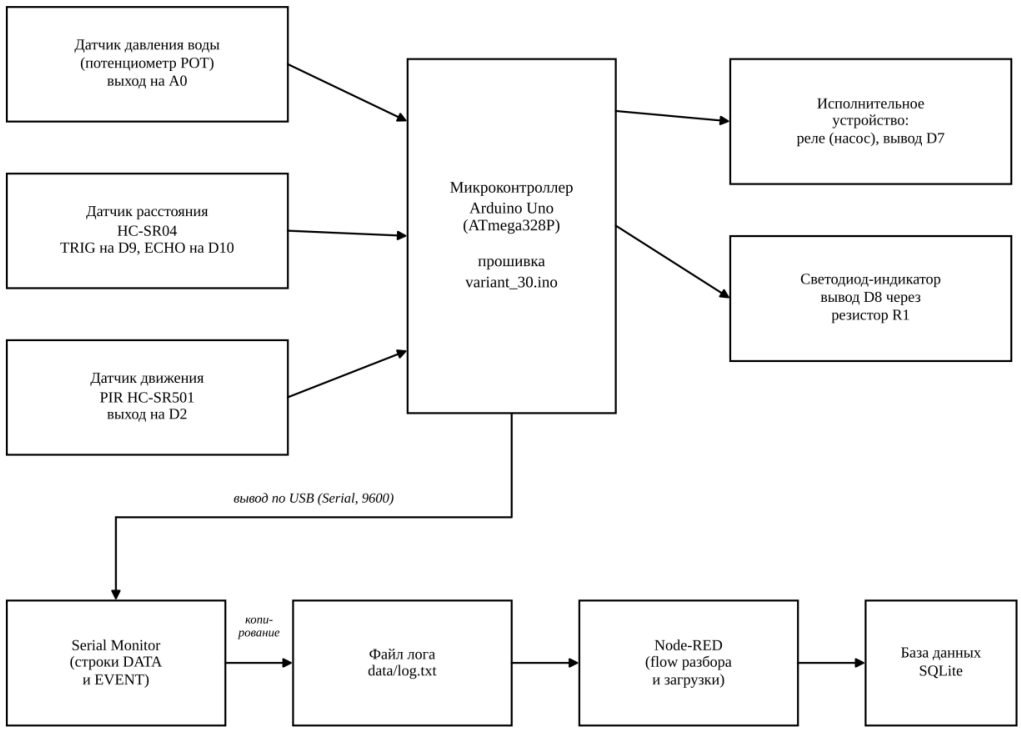


Рисунок 1 – Структурная схема системы

Структурная схема в увеличенном виде приведена в приложении А.

2.2. Алгоритм функционирования

Программа микроконтроллера выполняется циклически (рисунок 2). После инициализации порта и выводов реле и светодиод выключены, номер измерения n = 0, признак превышения порога alarm = ложь. В каждом цикле увеличивается номер измерения, считываются три датчика, и в порт передаётся строка DATA.

Затем проверяется условие события s1 < 3 бар. Событие формируется только в момент смены состояния: если условие выполнено, а признак alarm сброшен, включаются реле и светодиод и передаётся EVENT с типом THRESHOLD; если условие не выполнено, а alarm установлен, исполнительные устройства выключаются и передаётся EVENT с типом NORMAL. Такой подход исключает повторную отправку одного и того же события в каждом цикле. В конце цикла выдерживается пауза T = 2 с.

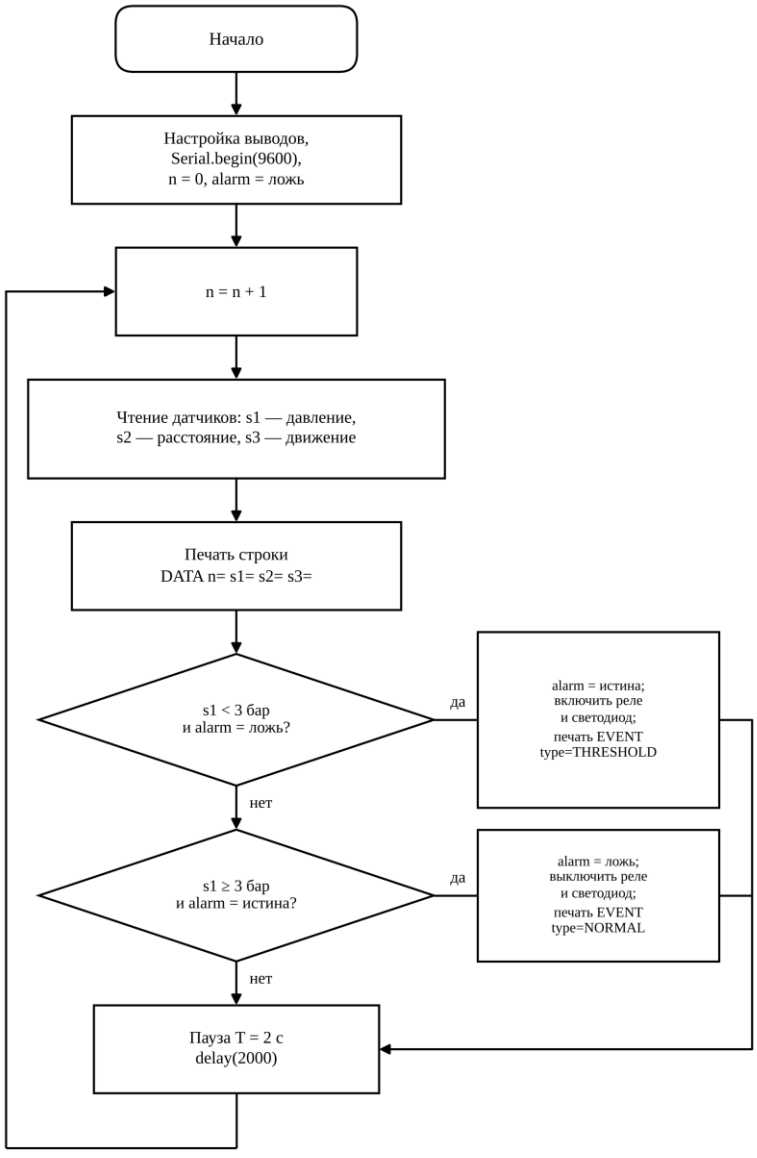


Рисунок 2 – Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках, которые одинаково читаются человеком и разбираются программой. Скорость порта – 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 – Формат строк последовательного порта

Тип	Формат	Пример
DATA	DATA n=<номер> s1=<давление> s2=<расстояние> s3=<движение>	DATA n=4 s1=2.00 s2=215.3 s3=1
EVENT	EVENT n=<номер> type=THRESHOLD | NORMAL	EVENT n=4 type=THRESHOLD

Время измерения в базе данных вычисляется по номеру: $t_s = n \cdot T$, где $T = 2$ с. Тип события FAULT программа Arduino не передаёт: его создаёт Node-RED, когда значение датчика отсутствует или выходит за допустимый диапазон (s1: 0–10 бар, s2: 2–400 см, s3: 0–1).

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты, которые есть в библиотеках EasyEDA и Wokwi: плата Arduino Uno R3 на микроконтроллере ATmega328P; потенциометр 10 кОм, имитирующий датчик давления; ультразвуковой дальномер HC-SR04; пассивный инфракрасный датчик движения HC-SR501 (PIR); одноканальный модуль реле 5 В; белый светодиод индикатора и зелёный светодиод, изображающий нагрузку насоса. Подключение к выводам платы приведено в таблице 3.

Таблица 3 – Подключение компонентов к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Давление воды (s1)	RP1, потенциометр 10 кОм	A0	аналоговый вход
TRIG (s2)	B1, HC-SR04	D9	выход
ECHO (s2)	B1, HC-SR04	D10	вход
Движение (s3)	B2, HC-SR501	D2	цифровой вход
Насос	K1, модуль реле (вход IN)	D7	выход
Индикатор	HL1, белый светодиод через R1	D8	выход
Питание	B1, B2, K1, RP1	5 V, GND	–

Потенциометр включён делителем между 5 В и общим проводом, движок подключён к A0. Десятиразрядный АЦП преобразует напряжение 0–5 В в код 0–1023, поэтому давление вычисляется по формуле $P = 10 \cdot код/1023$, а шаг измерения составляет $10/1023 \approx 0{,}0098$ бар. Порогу 3 бар соответствует код 307: при коде 306 давление равно 2,991 бар (ниже порога), при коде 307 – 3,001 бар (выше порога).

Дальномер HC-SR04 измеряет время прохождения ультразвукового импульса до объекта и обратно. Расстояние в сантиметрах $L = t \cdot 0{,}034/2$, где t – длительность эхо-сигнала в микросекундах, 0,034 см/мкс – скорость звука. Максимальному расстоянию 400 см соответствует время 23,5 мс, поэтому ожидание эха ограничено 30 мс; если эхо не получено, программа считает, что объекта нет, и возвращает 400 см.

Нагрузка насоса (зелёный светодиод HL2 с резистором R2) подключена к контактам реле: общий контакт COM соединён с питанием 5 В, нормально разомкнутый контакт NO – с анодом светодиода. При срабатывании реле нагрузка получает питание.

3.2. Расчёт резисторов и оценка потребления

Резистор R1 ограничивает ток белого светодиода HL1, подключённого к выводу D8 (выходное напряжение 5 В). Прямое падение напряжения белого светодиода принимаем равным Uпр = 3,2 В (значение из методических указаний), рабочий ток по заданию I = 20 мА:

$$R1 = (Uпит – Uпр) / I = (5 – 3,2) / 0,020 = 90 Ом.$$

Расчётное значение 90 Ом округляется вверх до ближайшего номинала из ряда – 100 Ом: так ток получается не больше заданного. Фактический ток I = (5 – 3,2) / 100 = 18 мА. Мощность, рассеиваемая резистором, $P = I^2 \cdot R = 0,018^2 \cdot 100 \approx 0,032$ Вт, достаточно резистора 0,25 Вт. Ток 18 мА не превышает допустимых 40 мА для вывода ATmega328P.

Для зелёного светодиода HL2 (Uпр = 2,2 В) принят резистор R2 = 220 Ом:

$$I = (5 – 2,2) / 220 \approx 12,7 мА; P = 0,0127^2 \cdot 220 \approx 0,035 Вт.$$

Оценка потребления от шины 5 В приведена в таблице 4. Суммарный ток около 168 мА, что меньше 500 мА, допустимых для порта USB, поэтому внешний источник питания не требуется.

Таблица 4 – Оценка потребляемого тока

Потребитель	**Ток, мА**	**Примечание**
Плата Arduino Uno (ATmega328P)	50	типовое
Дальномер HC-SR04	15	по документации
Датчик движения HC-SR501	0,05	< 65 мкА
Модуль реле, катушка	72	при включении
Светодиод HL1 с резистором R1	18	расчёт
Светодиод HL2 с резистором R2	12,7	расчёт
Потенциометр RP1 (5 В / 10 кОм)	0,5	расчёт
Итого	**≈ 168**	< 500 (USB)

3.3. Принципиальная схема в EasyEDA

Схема электрическая принципиальная выполнена в EasyEDA на листе формата А4 с основной надписью по ГОСТ 2.104-2006 (форма 1). Условные графические обозначения соответствуют ГОСТ 2.721-74 и ГОСТ 2.730-73, позиционные обозначения – ГОСТ 2.710-81: A1 – плата Arduino Uno, B1 – дальномер, B2 – датчик движения, HL1, HL2 – светодиоды, K1 – реле, R1, R2 – резисторы, RP1 – потенциометр. Схема приведена в приложении Б, перечень элементов – в приложении В.

Порядок построения схемы: сначала в схему вводятся компоненты из библиотеки и задаются позиционные обозначения; затем проводятся цепи питания (5 V, GND) и сигнальные цепи в соответствии с таблицей 3; каждая цепь получает сетевую метку; после этого выполняется проверка правил (Design Manager → ERC), и схема экспортируется в PNG для вставки в рамку.

3.4. Модель в Wokwi

После утверждения схемы выполнена модель в симуляторе Wokwi (приложение Г). В проект добавлены элементы wokwi-arduino-uno, wokwi-potentiometer, wokwi-hc-sr04, wokwi-pir-motion-sensor, wokwi-relay-module, wokwi-led (два) и два резистора. Соединения повторяют схему EasyEDA. Давление задаётся движком потенциометра, расстояние – полем «Distance» дальномера, движение – ручным переключением датчика PIR.

Wokwi работает в браузере и не может записывать файлы на диск компьютера. Поэтому строки, выводимые в Serial Monitor, копируются и сохраняются в файл data/log.txt вручную; дальнейшая загрузка в базу данных выполняется автоматически (раздел 6).

4. Разработка прошивки

Программа для Arduino Uno написана на языке C++ в среде Wokwi; полный текст приведён в приложении Д. Параметры варианта вынесены в константы в начале программы: номера выводов, период опроса T_MS = 2000 мс, диапазон датчика давления S1_MIN = 0 и S1_MAX = 10, порог THRESHOLD = 3,0 бар. Для другого варианта достаточно изменить эти константы и три функции чтения датчиков.

Структура программы определяется набором функций (таблица 5).

Таблица 5 – Функции программы

Функция	Назначение
setup()	настройка порта (9600 бод) и режимов выводов, начальное выключение реле и светодиода
loop()	основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T
readS1()	чтение давления: analogRead(A0), пересчёт кода 0–1023 в диапазон 0–10 бар
readS2() / readDistance()	запуск HC-SR04 импульсом 10 мкс, измерение эха pulseIn(), пересчёт в сантиметры
readS3()	чтение датчика движения digitalRead(D2)
actuatorOn() / actuatorOff()	включение и выключение реле насоса
conditionMet()	условие события: s1 < THRESHOLD

Для передачи данных использованы функции Serial.print() и Serial.println(): давление выводится с двумя знаками после запятой, расстояние – с одним, движение – целым числом. Такой формат совпадает с протоколом из таблицы 2 и разбирается в Node-RED без дополнительных преобразований.

Пауза между измерениями реализована функцией delay(T_MS). Для данной задачи это допустимо: нет других действий, которые нужно выполнять в промежутке между опросами, а точность периода в пределах долей секунды достаточна для расчёта времени $t_s = n \cdot T$.

5. Проектирование базы данных

5.1. Модель данных

Для хранения данных выбрана встраиваемая СУБД SQLite: база целиком хранится в одном файле db/variant.db, не требует сервера и работает под управлением консольной программы sqlite3. Модель состоит из двух таблиц (рисунок 3): readings – показания датчиков, events – события и неисправности.

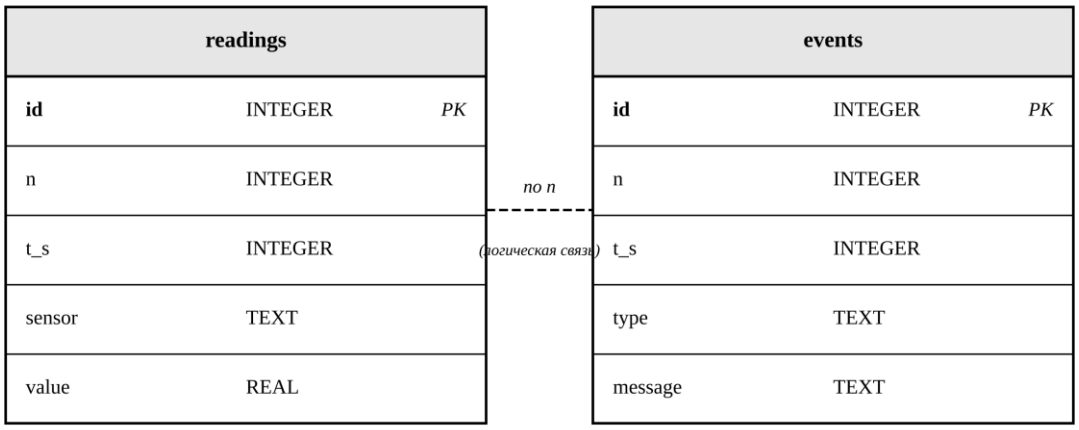


Рисунок 3 – Модель данных

Показания хранятся в «длинном» формате: один датчик в одном цикле – одна строка. Благодаря этому структура таблицы не зависит от числа датчиков, а запросы по конкретному датчику выполняются условием по полю sensor. Связь между таблицами логическая: по номеру измерения n.

5.2. Описание таблиц

Таблица 6 – Структура таблицы readings

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения из журнала
t_s	INTEGER, NOT NULL	время от начала измерений, с (t_s = n·T)
sensor	TEXT, NOT NULL	датчик: s1, s2 или s3
value	REAL, NOT NULL	значение датчика

Таблица 7 – Структура таблицы events

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения, при котором возникло событие
t_s	INTEGER, NOT NULL	время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	пояснение (для FAULT – какой датчик неисправен)

Скрипт создания таблиц db/schema.sql использует оператор CREATE TABLE IF NOT EXISTS, поэтому его можно выполнять повторно без ошибок. Текст скрипта приведён в приложении Ж.

5.3. Контрольные запросы

Для проверки данных составлены три запроса (файл db/queries.sql, приложение Ж). Запрос 1 выводит десять последних показаний и показывает, что загрузка дошла до конца журнала. Запрос 2 вычисляет минимум, среднее и максимум по каждому датчику с помощью агрегатных функций MIN, AVG, MAX и группировки GROUP BY sensor; результат позволяет быстро заметить значения, не похожие на реальные. Запрос 3 выводит все события по порядку их возникновения.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Загрузка данных выполнена потоком Node-RED, показанным на рисунке 4. Поток следит за файлом data/log.txt и при каждом сохранении файла автоматически перезагружает базу данных. Дополнительно предусмотрена кнопка ручного запуска. Назначение узлов приведено в таблице 8.

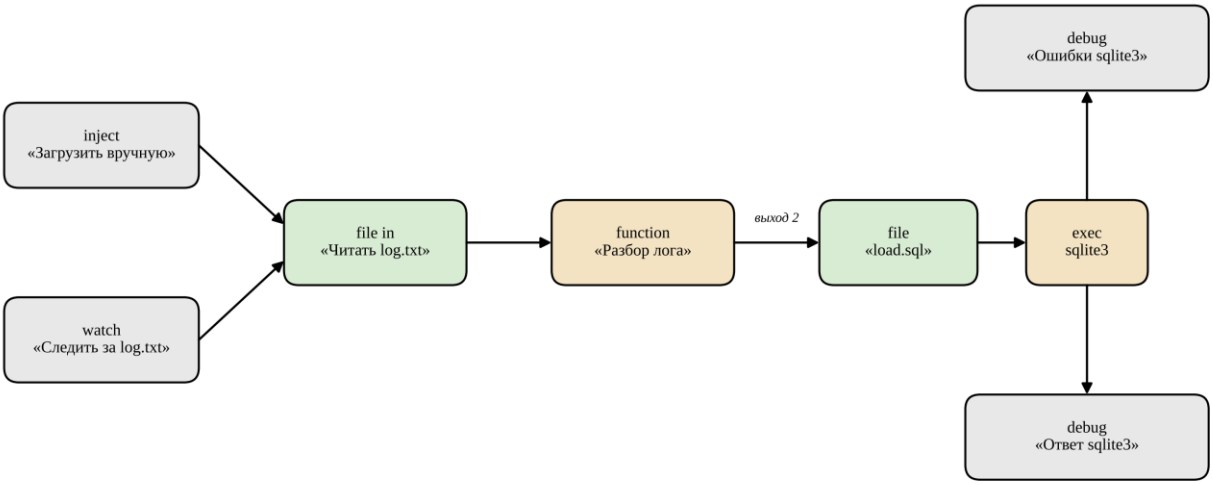


Рисунок 4 – Поток Node-RED

Таблица 8 – Узлы потока

Узел	Тип	Настройка и назначение
Следить за data/log.txt	watch	формирует сообщение при изменении файла журнала
Загрузить лог вручную	inject	ручной запуск потока кнопкой
Читать data/log.txt	file in	читает файл целиком одной строкой (utf8)
Разбор лога	function	разбирает строки, проверяет диапазоны, формирует SQL; два выхода
Записать db/load.sql	file	пишет SQL-скрипт загрузки, режим «перезаписать файл»
Загрузить в базу	exec	sqlite3 db/variant.db ".read db/schema.sql" ".read db/load.sql"; флажок Append msg.payload снят
Ответ sqlite3	debug	вывод штатного ответа программы
Ошибки sqlite3	debug	вывод ошибок (второй и третий выходы exec)

6.2. Функция разбора журнала

Функция (приложение Е) получает текст журнала и делит его на строки. Каждая строка разбивается по пробелам на тип сообщения (DATA или EVENT) и пары «ключ=значение». Номер измерения n преобразуется в число, время вычисляется как t_s = n·2. Для строки DATA для каждого из датчиков s1, s2, s3 формируется команда INSERT в таблицу readings. Значение проверяется по допустимому диапазону; если оно отсутствует или выходит за границы, добавляется команда INSERT в таблицу events с типом FAULT и текстом «датчик sN вне диапазона». Для строки EVENT формируется запись в events с типом, переданным контроллером.

Функция имеет два выхода. Первый выдаёт отдельные SQL-команды и оставлен для необязательного подключения узла sqlite. Второй выдаёт единый скрипт load.sql, который начинается с команд BEGIN и очистки таблиц (DELETE FROM readings, DELETE FROM events, сброс счётчиков sqlite_sequence) и заканчивается командой COMMIT. Очистка делает загрузку повторяемой: при повторной загрузке того же журнала записи не дублируются, а транзакция гарантирует, что база не останется в наполовину загруженном состоянии.

6.3. Запуск

Работа ведётся на компьютере без прав администратора, все программы портативные и лежат в папке %USERPROFILE%\tools. Командный файл start-nodered.bat перед запуском переходит в папку проекта, благодаря чему относительные пути data/log.txt и db/variant.db указываются от корня проекта. Порядок запуска: выполнить start-nodered.bat, открыть редактор по адресу http://localhost:1880, импортировать поток из файла nodered/flow.json и нажать «Deploy». После сохранения файла data/log.txt загрузка происходит автоматически; результат проверяется командой:

```
sqlite3 db/variant.db ".read db/queries.sql"
```

7. Тестирование и результаты

7.1. Штатный сценарий

Для проверки смоделирован заезд автомобиля на пост (12 циклов, 24 с). В циклах 1–2 автомобиля нет (эхо не получено, 400 см), давление в норме. В цикле 3 автомобиль въезжает на пост. В цикле 4 при включении воды давление падает до 2,00 бар – формируется событие THRESHOLD, включаются реле и светодиод. В цикле 8 давление восстанавливается до 4,40 бар – формируется событие NORMAL. В циклах 9–12 автомобиль выезжает. Журнал Serial Monitor приведён в таблице 9 и в приложении И.

Таблица 9 – Журнал штатного сценария

n	t_s, с	s1, бар	s2, см	s3	Событие
1	2	5,00	400,0	0	–
2	4	5,00	400,0	0	–
3	6	5,00	312,5	0	–
4	8	2,00	215,3	1	THRESHOLD
5	10	1,96	148,2	1	–
6	12	1,76	96,4	1	–
7	14	2,05	52,0	1	–
8	16	4,40	52,0	0	NORMAL
9	18	6,00	52,0	0	–
10	20	6,00	120,7	0	–
11	22	6,00	400,0	0	–
12	24	5,47	400,0	0	–

После сохранения журнала Node-RED загрузил данные в базу. Счётчики записей: в таблице readings 36 строк (12 циклов по 3 датчика), в таблице events – 2 строки. Результаты контрольных запросов приведены в таблицах 10–12.

Таблица 10 – Запрос 1: десять последних показаний

id	n	t_s	sensor	value
36	12	24	s3	0
35	12	24	s2	400
34	12	24	s1	5,47
33	11	22	s3	0

id	n	t_s	sensor	value
32	11	22	s2	400
31	11	22	s1	6
30	10	20	s3	0
29	10	20	s2	120,7
28	10	20	s1	6
27	9	18	s3	0

Таблица 11 – Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	1,76	4,22	6
s2	52	220,76	400
s3	0	0,33	1

Значения лежат в допустимых диапазонах: давление 1,76–6,00 бар, расстояние 52–400 см, движение 0–1. Среднее значение s3 равно 0,33: движение зафиксировано в 4 циклах из 12.

Таблица 12 – Запрос 3: все события

id	n	t_s	type	message
1	4	8	THRESHOLD	–
2	8	16	NORMAL	–

События совпали с моделируемой ситуацией: THRESHOLD в цикле 4 (t_s = 8 с) и NORMAL в цикле 8 (t_s = 16 с). Неисправностей FAULT нет.

7.2. Сценарий неисправности датчика

Для проверки контроля диапазонов в журнал подана строка, в которой датчик давления передаёт недопустимое значение 999 (диапазон 0–10 бар):

```
DATA n=1 s1=999 s2=120.0 s3=0
```

После загрузки в таблице readings появились 3 строки (по одной на каждый датчик), а в таблице events – 1 строка: n = 1, t_s = 2, type = FAULT, message = «датчик s1 вне диапазона». Данные других датчиков (s2 = 120 см, s3 = 0) сохранены в штатном порядке. Файл load.sql для этого теста приведён в приложении И.

7.3. Повторная загрузка

Журнал штатного сценария сохранён повторно без изменений. Счётчики остались прежними: 36 строк в readings и 2 строки в events. Это подтверждает, что очистка таблиц в начале скрипта load.sql предотвращает дублирование данных.

7.4. Сводка результатов

Таблица 13 – Результаты проверок

№	Проверка	Ожидаемый результат	Получено	Итог
1	Загрузка штатного журнала	36 показаний, 2 события	36 и 2	пройден
2	Событие при s1 < 3 бар	THRESHOLD при n = 4	n = 4, t_s = 8	пройден
3	Возврат давления в норму	NORMAL при n = 8	n = 8, t_s = 16	пройден
4	Значение вне диапазона	FAULT, 3 показания	1 FAULT, 3 показания	пройден
5	Повторная загрузка	счётчики не растут	36 и 2	пройден
6	Работа K1 и HL1 в Wokwi	включены при s1 < 3 бар	включались и выключались по событиям	пройден

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T = 2 с приведён в таблице 14.

Таблица 14 – Объём данных

Показатель	Расчёт	Результат
Циклов в сутки	86 400 с / 2 с	43 200
Записей readings в сутки	43 200 · 3 датчика	129 600
Размер строки в базе (оценка)	id, n, t_s, sensor, value	≈ 40 Б
Объём базы за сутки	129 600 · 40 Б	≈ 5,2 МБ
Размер строки журнала	DATA n=43200 s1=5.00 s2=400.0 s3=0 + CRLF	≈ 36 Б
Объём журнала за сутки	43 200 · 36 Б	≈ 1,6 МБ
Объём базы за 30 суток	5,2 МБ · 30	≈ 156 МБ
Объём базы за год	5,2 МБ · 365	≈ 1,9 ГБ

Объём базы за год близок к 2 ГБ, что для SQLite не является проблемой: ограничения СУБД намного выше, а практический предел определяется свободным местом на диске. При длительной эксплуатации рекомендуется архивировать журнал и периодически удалять показания старше заданного срока.

9. Управление версиями Git

Проект ведётся в локальном репозитории Git. Структура папки проекта: firmware (программа Arduino), db (скрипты базы данных и запросы), nodered (экспорт потока), data (журнал), sim (модель Wokwi и скриншоты), docs (схема EasyEDA, пояснительная записка), файлы start-nodered.bat, env.bat и README.md. Файл базы db/variant.db и промежуточный db/load.sql создаются автоматически, поэтому в репозиторий не включаются (файл .gitignore).

Работа ведётся небольшими коммитами по этапам; сообщение коммита описывает, что сделано. Порядок работы:

```
git init
git config user.name "Иванов Иван"
git config user.email "ivanov@example.com"
git add .
git commit -m "Структура проекта"
... (по одному коммиту на этап)
git tag v1.0
git log --oneline
```

История коммитов проекта приведена в таблице 15 и в приложении К. Метка v1.0 поставлена на финальной версии, сданной на защиту.

Таблица 15 – Коммиты проекта

Хеш	Дата	Сообщение
a1f3c90	03.10.2026	Структура проекта, README, .gitignore
b72d5e1	04.10.2026	Схема электрическая принципиальная в EasyEDA
c8e0a47	05.10.2026	Модель Wokwi и прошивка variant_30.ino
d49b2f8	06.10.2026	Схема БД schema.sql и контрольные запросы
e5c61d3	07.10.2026	Поток Node-RED: разбор и загрузка журнала
f0a97b2	08.10.2026	Журнал тестов и результаты запросов
0b3e8d6	09.10.2026	Пояснительная записка, метка v1.0

Выводы

В ходе курсового проекта разработана и проверена виртуальная система сбора и учёта данных для сценария «Автомойка» (вариант №30). Получены следующие результаты:

- выполнена структурная схема и алгоритм работы, определён текстовый протокол обмена с контроллером;
- подобраны компоненты, рассчитаны резисторы R1 = 100 Ом и R2 = 220 Ом, суммарное потребление около 168 мА не превышает возможностей порта USB;
- принципиальная схема выполнена в EasyEDA по требованиям ЕСКД, модель собрана в Wokwi;
- написана программа для Arduino Uno, которая опрашивает три датчика каждые 2 с и формирует события при давлении ниже 3 бар только в момент смены состояния;
- спроектирована база данных SQLite из двух таблиц и подготовлены три контрольных запроса;
- настроен поток Node-RED, автоматически и повторяемо загружающий журнал в базу и фиксирующий неисправности датчиков;
- тестирование на штатном и аварийном сценариях подтвердило правильность работы: 36 показаний и 2 события в штатном режиме, событие FAULT при недопустимом значении;
- оценён объём данных – около 5,2 МБ в сутки, проект оформлен в репозитории Git.

Разработанный проект может быть использован как основа для других сценариев: при смене датчиков достаточно изменить константы и функции чтения в программе, диапазоны и период в функции Node-RED, а структура базы данных остаётся прежней.

Список используемых сокращений

АЦП	аналого-цифровой преобразователь
БД	база данных
ГОСТ	государственный стандарт
ЕСКД	**единая система конструкторской документации**
ИИС	**интеллектуальные интегрированные системы**
СУБД	**система управления базами данных**
ERC	**проверка электрических правил (Electrical Rules Check)**
PIR	**пассивный инфракрасный датчик движения (Passive Infrared)**
SQL	**язык структурированных запросов (Structured Query Language)**
UART	**универсальный асинхронный приёмопередатчик**
USB	**универсальная последовательная шина**

Список используемой литературы

1. Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. – СПб.: БХВ-Петербург, 2016. – 320 с.

2. Монк С. Программируем Arduino. Профессиональная работа со скетчами. – СПб.: Питер, 2017. – 288 с.

3. Чакон С., Штрауб Б. Pro Git. – 2-е изд. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 01.10.2026).

4. ГОСТ 2.104-2006. ЕСКД. Основные надписи. – М.: Стандартинформ, 2007.

5. ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению. – М.: Стандартинформ, 2009.

6. ГОСТ 2.710-81. ЕСКД. Обозначения буквенно-цифровые в электрических схемах. – М.: Издательство стандартов, 1981.

7. ГОСТ 2.721-74. ЕСКД. Обозначения условные графические в схемах. Обозначения общего применения. – М.: Издательство стандартов, 1974.

8. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.

9. Документация Arduino: Language Reference. – URL: https://www.arduino.cc/reference/en/ (дата обращения: 01.10.2026).

10. Документация Wokwi. – URL: https://docs.wokwi.com/ (дата обращения: 01.10.2026).

11. Документация Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 01.10.2026).

12. Документация SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 01.10.2026).

13. Документация EasyEDA. – URL: https://docs.easyeda.com/ (дата обращения: 01.10.2026).

14. Микроконтроллер ATmega328P. Datasheet. – Microchip Technology Inc. – URL: https://www.microchip.com (дата обращения: 01.10.2026).

Приложение А Структурная схема

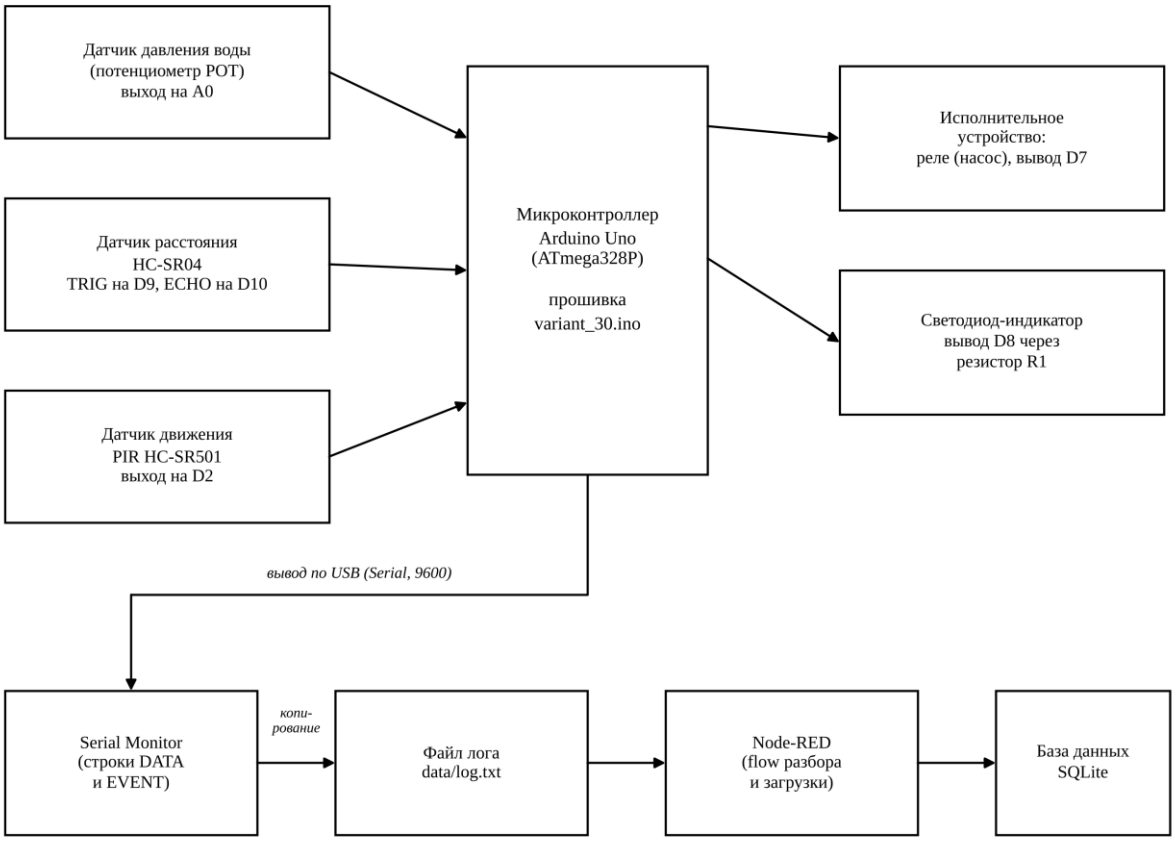
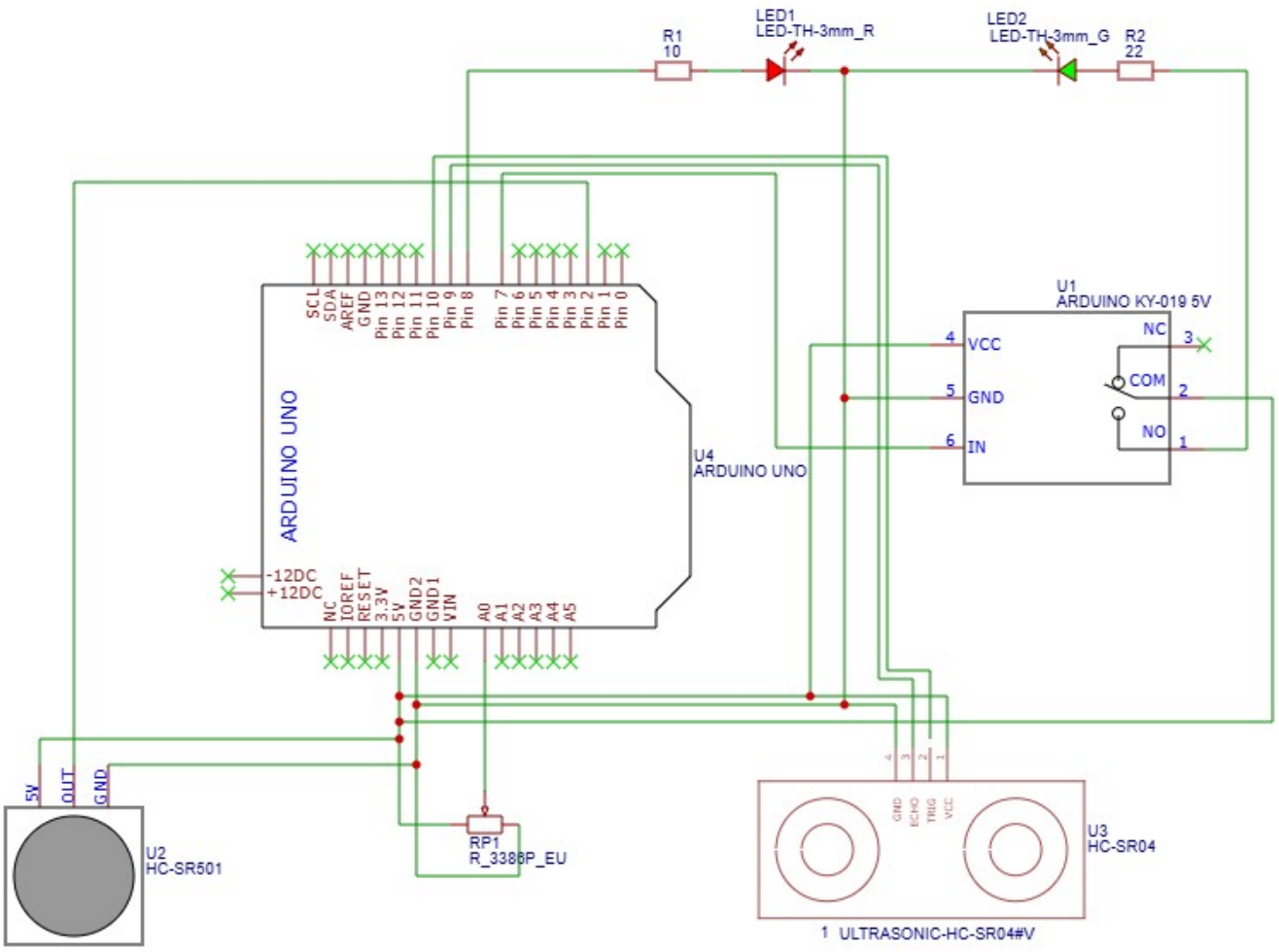


Рисунок А.1 – Структурная схема системы контроля поста автомобильной мойки



					КП.5415.30.ЭЗ
Изм.	Лист	№ докум.	Подп.	Дата	
Разраб.		Иванов И.И		01.10.26	Модуль управления Автомойкой
Пров.		Борисов Е.Н		01.10.26	
Н. контр.		Парадина Г.А		05.11.26	
Утв.		Бычков А.Ю		05.11.26	

Лит.	Лист	Листов
У	1	2

СПб ГБПОУ

Подп. и дата | Инв. № дубл. | Взам. Инв. № | Подп. и дата | Инв. № подл.

Поз. обозначение	Наименование	Кол.	Примечание
	Диоды		
LED1	Светодиод LED-TH-3mm красный	1	
LED2	Светодиод LED-TH-3mm зеленый	1	
	Резисторы		
R1	Резистор 10 Ом	1	
R2	Резистор 22 Ом	1	
RP1	Резистор подстроечный R_3386P_E	1	
	Модули и платы		
U1	Модуль реле KY-019 5V	1	
U2	Датчик движения HC-SR501	1	
U3	Ультразвуковой датчик HC-SR04	1	
U4	Плата расширения Arduino Uno Shield	1	

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата

КП.5415.30.33 ПЭ3

Разраб.	Иванов И.И	
Пров.	Борисов Е.Н	
Нач. сект.		
Н. контр.	Парадина Г.А	
Утв.	Бычков А.Ю	

Модуль управления Автомойкой

Перечень элементов

Лит.	Лист	Листов
у	2	2

СПб ГБПОУ

Приложение Г Модель в Wokwi

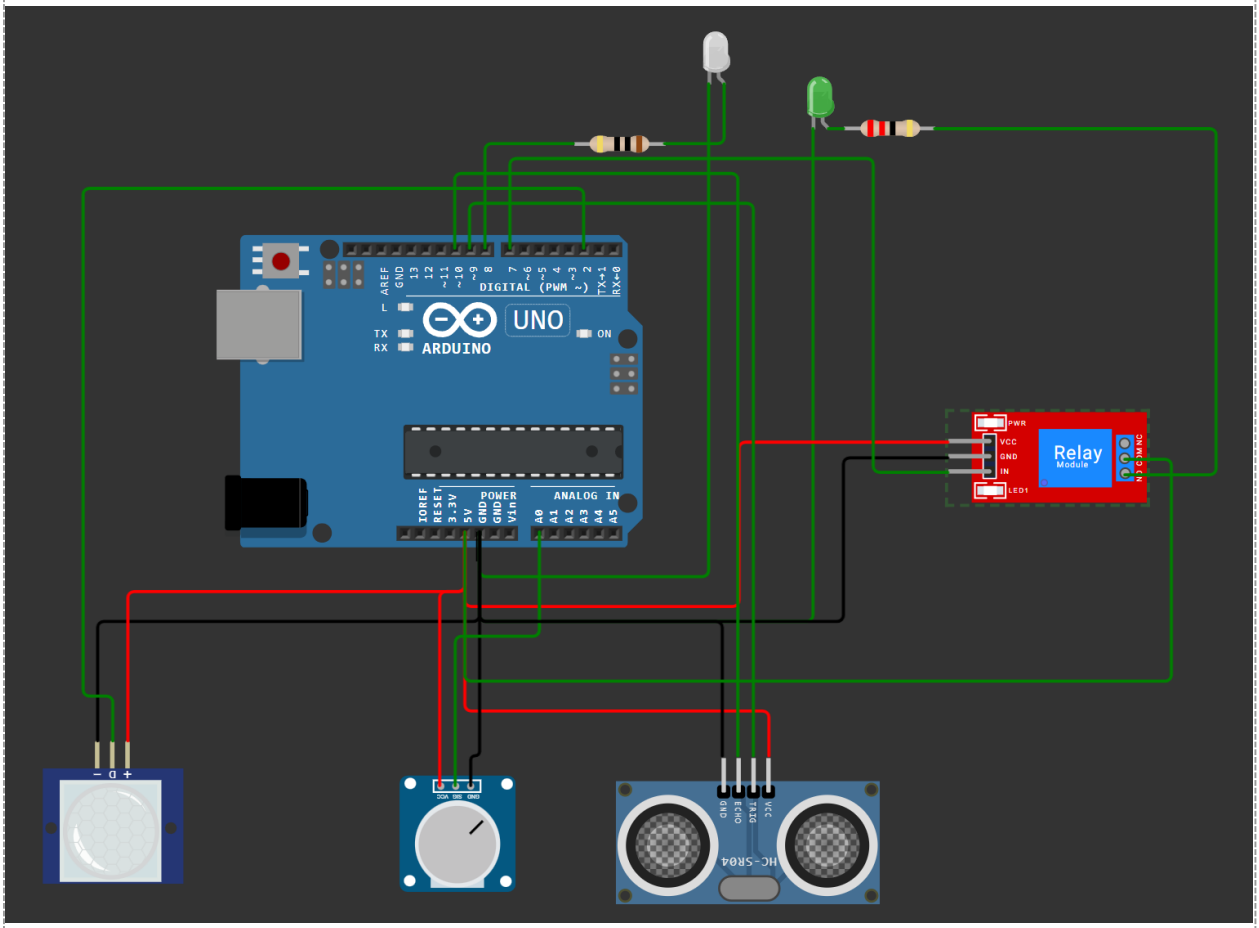


Рисунок Г.1 – Модель системы в симуляторе Wokwi

Приложение Д Листинг программы Arduino (variant_30.ino)

```
// Вариант 30. Автомойка. Система контроля давления воды
// Плата: Arduino Uno (ATmega328P), симулятор Wokwi
// Подключение:
//   Датчик 1 (POT, давление воды 0-10 бар)   -> A0
//   Датчик 2 (HC-SR04, расстояние до авто)   -> TRIG D9, ECHO D10
//   Датчик 3 (PIR, движение)                 -> D2
//   Реле K1 (насос)                          -> D7
//   Светодиод HL1 (белый) через R1 = 100 Ом  -> D8


const int PIN_S1 = A0;  // датчик 1: POT (давление воды)
const int PIN_TRIG = 9;   // датчик 2: HC-SR04, TRIG
const int PIN_ECHO = 10;  // датчик 2: HC-SR04, ECHO
const int PIN_S3 = 2;  // датчик 3: PIR (движение)
const int PIN_RELAY = 7;   // насос
const int PIN_LED = 8;    // светодиод-индикатор


const unsigned long T_MS = 2000;  // период опроса T = 2 с
const float S1_MIN = 0.0;   // диапазон датчика 1: 0...10
const float S1_MAX = 10.0;
const float THRESHOLD = 3.0;   // порог: давление < 3 бар


unsigned long n = 0;   // номер измерения
bool alarm = false;    // порог сейчас превышен?


float readDistance() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  long us = pulseIn(PIN_ECHO, HIGH, 30000);  // ждём эхо не дольше 30 мс
  if (us == 0) return 400.0;                 // эха нет: объект далеко
  return us * 0.034 / 2.0;                   // время эха -> расстояние, см
}


// Датчик 1: POT (давление воды)
float readS1() {
  return S1_MIN + (S1_MAX - S1_MIN) * analogRead(PIN_S1) / 1023.0;
}


// Датчик 2: US (расстояние до автомобиля)
float readS2() {
  return readDistance();
}


// Датчик 3: PIR (движение)
float readS3() {
  return digitalRead(PIN_S3);
}


void actuatorOn() {
  digitalWrite(PIN_RELAY, HIGH);
}


void actuatorOff() {
  digitalWrite(PIN_RELAY, LOW);
}


bool conditionMet(float s1, float s2, float s3) {
  return s1 < THRESHOLD;
}


void setup() {
  Serial.begin(9600);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_S3, INPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
```

```
  actuatorOff();
  digitalWrite(PIN_LED, LOW);
}

void loop() {
  n++;
  float s1 = readS1();
  float s2 = readS2();
  float s3 = readS3();


  Serial.print("DATA n=");  Serial.print(n);
  Serial.print(" s1=");     Serial.print(s1, 2);
  Serial.print(" s2=");     Serial.print(s2, 1);
  Serial.print(" s3=");     Serial.println((int)s3);


  bool cond = conditionMet(s1, s2, s3);


  if (cond && !alarm) {
    alarm = true;
    actuatorOn();
    digitalWrite(PIN_LED, HIGH);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.println(" type=THRESHOLD");
  } else if (!cond && alarm) {
    alarm = false;
    actuatorOff();
    digitalWrite(PIN_LED, LOW);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.println(" type=NORMAL");
  }


  delay(T_MS);
}
```

Приложение Е Листинг функции разбора журнала Node-RED

```
// msg.payload — весь текст лога одной строкой
const T = 2;                                    // период опроса, с
const RANGES = { s1: [0, 10], s2: [2, 400], s3: [0, 1] };


const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];


text.split(/\r?\n/).forEach(raw => {
  const line = raw.trim();
  if (!line) return;


  // строка вида DATA n=3 s1=2.40: делим по пробелам, затем по знаку =
  const parts = line.split(' ');
  const kind = parts[0];                        // DATA или EVENT
  const f = {};
  parts.slice(1).forEach(p => {
    const kv = p.split('=');
    f[kv[0]] = kv[1];
  });


  const n = parseInt(f.n);
  if (isNaN(n)) return;
  const t = n * T;


  if (kind === 'DATA') {
    ['s1', 's2', 's3'].forEach(s => {
      const v = parseFloat(f[s]);
      if (!isNaN(v)) {
        sql.push(`INSERT INTO readings(n,t_s,sensor,value)
VALUES(${n},${t},'${s}',${v});`);
      }
      if (isNaN(v) || v < RANGES[s][0] || v > RANGES[s][1]) {
        sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'FAULT','датчик
${s} вне диапазона');`);
      }
    });
  } else if (kind === 'EVENT') {
    sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'${f.type}','');`);
  }
});


// выход 1: по одной команде (для необязательного узла sqlite)
const toSqlite = sql.map(s => ({ topic: s.slice(0, -1) }));
// выход 2: один файл load.sql (сначала очищает таблицы, поэтому повторная загрузка не
дублирует данные)
const head = 'BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\n' +
  "DELETE FROM sqlite_sequence WHERE name IN ('readings','events');\n";
const toFile = { payload: head + sql.join('\n') + '\nCOMMIT;\n' };
return [toSqlite, toFile];
```

Приложение Ж Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
  id     INTEGER PRIMARY KEY AUTOINCREMENT,
  n      INTEGER NOT NULL,
  t_s    INTEGER NOT NULL,
  sensor TEXT    NOT NULL,
  value  REAL    NOT NULL
);


CREATE TABLE IF NOT EXISTS events (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  n       INTEGER NOT NULL,
  t_s     INTEGER NOT NULL,
  type    TEXT    NOT NULL,
  message TEXT
);
```

db/queries.sql

```
-- 1. 10 последних показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;


-- 2. минимум, среднее и максимум по каждому датчику
SELECT sensor, MIN(value) AS min_v, ROUND(AVG(value), 2) AS avg_v, MAX(value) AS max_v
FROM readings GROUP BY sensor;


-- 3. все события по порядку
SELECT * FROM events ORDER BY id;
```

Приложение И Журнал Serial и загрузочный скрипт

data/log.txt (штатный сценарий)

```
DATA n=1 s1=5.00 s2=400.0 s3=0
DATA n=2 s1=5.00 s2=400.0 s3=0
DATA n=3 s1=5.00 s2=312.5 s3=0
DATA n=4 s1=2.00 s2=215.3 s3=1
EVENT n=4 type=THRESHOLD
DATA n=5 s1=1.96 s2=148.2 s3=1
DATA n=6 s1=1.76 s2=96.4 s3=1
DATA n=7 s1=2.05 s2=52.0 s3=1
DATA n=8 s1=4.40 s2=52.0 s3=0
EVENT n=8 type=NORMAL
DATA n=9 s1=6.00 s2=52.0 s3=0
DATA n=10 s1=6.00 s2=120.7 s3=0
DATA n=11 s1=6.00 s2=400.0 s3=0
DATA n=12 s1=5.47 s2=400.0 s3=0
```

Тест неисправности: строка журнала

```
DATA n=1 s1=999 s2=120.0 s3=0
```

db/load.sql, сформированный функцией для теста неисправности

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s1',999);
INSERT INTO events(n,t_s,type,message) VALUES(1,2,'FAULT','датчик s1 вне диапазона');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s2',120);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s3',0);
COMMIT;
```

Приложение К История Git

git log --oneline

```
0b3e8d6 (HEAD -> main, tag: v1.0) Пояснительная записка, метка v1.0
f0a97b2 Журнал тестов и результаты запросов
e5c61d3 Поток Node-RED: разбор и загрузка журнала
d49b2f8 Схема БД schema.sql и контрольные запросы
c8e0a47 Модель Wokwi и прошивка variant_30.ino
b72d5e1 Схема электрическая принципиальная в EasyEDA
a1f3c90 Структура проекта, README, .gitignore
```